

# Smart Hostel Management System

Academic Project Report — Java-Based Console Application

---

**Programming Language:** Java

**Application Type:** Console-Based Application

**Main Source File:** Main.java

**Data Storage:** Text Files (students.txt, rooms.txt, complaints.txt, fees.txt, notices.txt)

**Submitted by:** Saumya Rani

**Reg Number:** 25BAI11014

**Academic Year:** 2026–2027

**Department:** Computer Science & Engineering



# Introduction, Problem Statement & Objectives

## Introduction

Hostel management in educational institutions involves coordinating student accommodations, fee collections, maintenance complaints, and administrative communications. Traditional manual methods are error-prone and inefficient. The **Smart Hostel Management System (SHMS)** is a Java-based console application that digitizes these operations through a structured, file-backed system.

The application uses a single `Main.java` source file with text-file persistence, eliminating the need for external databases. Two user roles — **Administrator** and **Student** — interact through authenticated sessions with role-specific dashboards.

## Problem Statement

Manual hostel administration faces these challenges:

- No centralized student accommodation records
- Room allocation errors leading to overbooking
- Delayed complaint resolution with no tracking
- Fragmented fee records causing reconciliation issues
- No systematic notice dissemination
- Lack of consolidated administrative reports

## Project Objectives

- Digitize student, room, complaint, fee, and notice management
- Implement smart room allocation based on current available-bed information.
- Provide role-based access for Admin and Student
- Persist data using structured text files
- Demonstrate core Java OOP concepts in a practical system

# Functional & Non-Functional Requirements

## Functional Requirements

### Administrator Module

| Module               | Operations                                                                                   |
|----------------------|----------------------------------------------------------------------------------------------|
| Student Management   | View all, Add, Search (by ID and by name), Update, Delete                                    |
| Room Management      | View, Add, Smart Allocation, Specific Allocation, Vacate, Check Availability, Track Capacity |
| Complaint Management | View all, View pending, Update status (Pending / In Progress / Resolved), Delete             |
| Fee Management       | View records, Add records, Update payments, Search by Student ID                             |
| Notice Management    | View, Add, Delete notices                                                                    |
| Reports              | Students, Rooms, Beds, Complaints, Fees, Notices summary                                     |

### Student Module

| Feature        | Operations                                         |
|----------------|----------------------------------------------------|
| Authentication | Login via Student ID, Logout                       |
| Profile & Room | View personal profile, View room details           |
| Complaints     | Submit complaints, View personal complaint history |
| Fees           | View fee status, Make payments                     |
| Notices        | View all active hostel notices                     |

# Non-Functional Requirements

## Performance

File read/write operations in very less time

## Reliability

Graceful exception handling for file I/O errors, null pointers, and invalid user inputs.

## Security

Credential-based authentication. Admin: admin / 1234. Students authenticate via Student ID.

## Maintainability

Single-file architecture with modular method decomposition for straightforward maintenance.

## Portability

Runs on any platform with a Java Runtime Environment (JRE 8 or later).

## Scalability

ArrayList-based in-memory structures support dynamic record growth without fixed limits.

# System Architecture & Design Decisions

## System Architecture

The SHMS follows a **three-layer architecture** within a single Java source file:

- **Presentation Layer:** Console-based menus and input/output via `Scanner`
- **Business Logic Layer:** Java classes encapsulating operations for each entity
- **Data Layer:** Text files (`students.txt` , `rooms.txt` , etc.) for persistent storage

At startup, all records load into `ArrayList` collections. Operations execute in memory, and changes persist to disk upon completion.

## Design Decisions & Rationale

### Single-File Architecture

All classes in `Main.java` simplify compilation and submission without sacrificing modularity.

### Text File Persistence

Plain text files eliminate database dependencies while remaining human-readable for debugging.

### Inheritance via Person Class

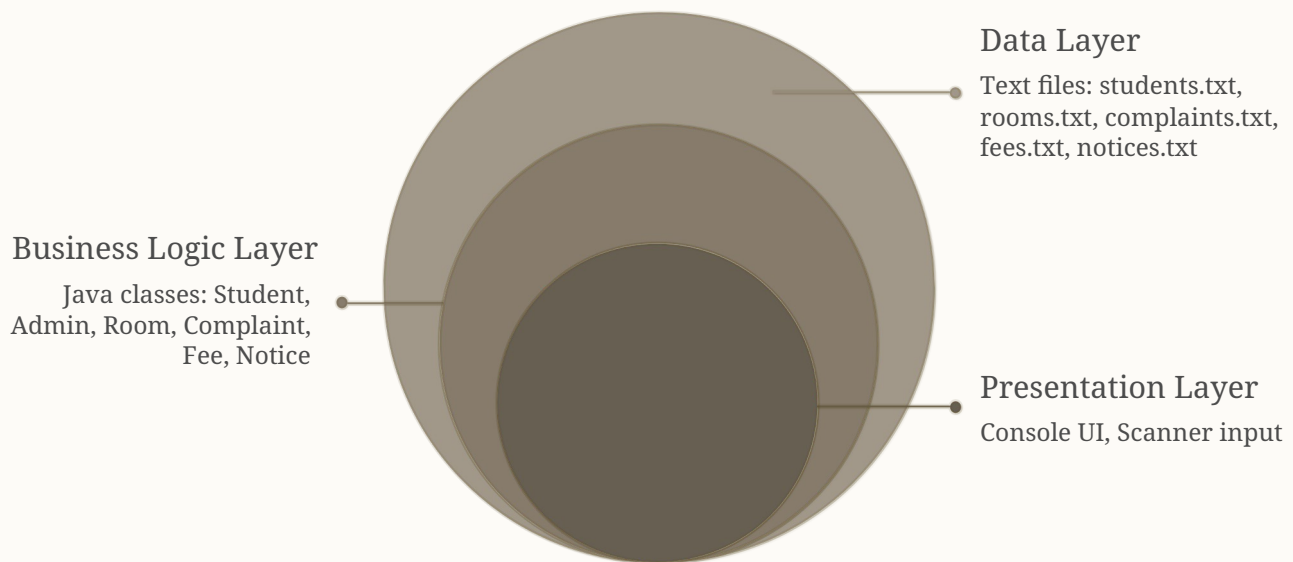
`Student` and `Admin` extend `Person` , eliminating redundant attributes like name and phone.

### Displayable Interface

Enforces consistent `displayInfo()` across all entity classes for uniform console output.

# Architecture Diagram

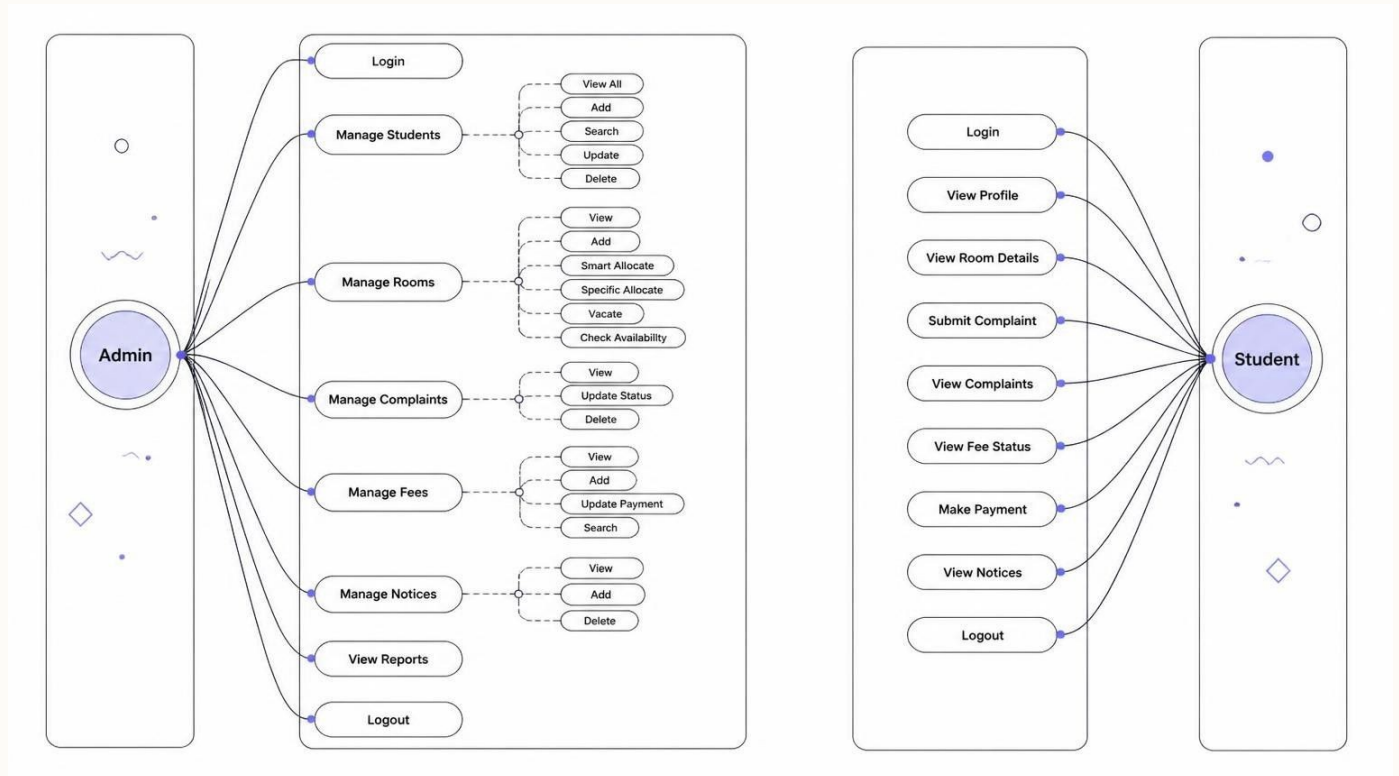
The Smart Hostel Management System (SHMS) employs a three-layered architecture to organize its components and facilitate a clear separation of concerns, even within a single Java source file. This design ensures that user interaction, core system logic, and data storage are managed distinctly.



This layered architecture ensures separation of concerns despite the single-file constraint, enabling clean data flow from user input through business logic to persistent file storage.

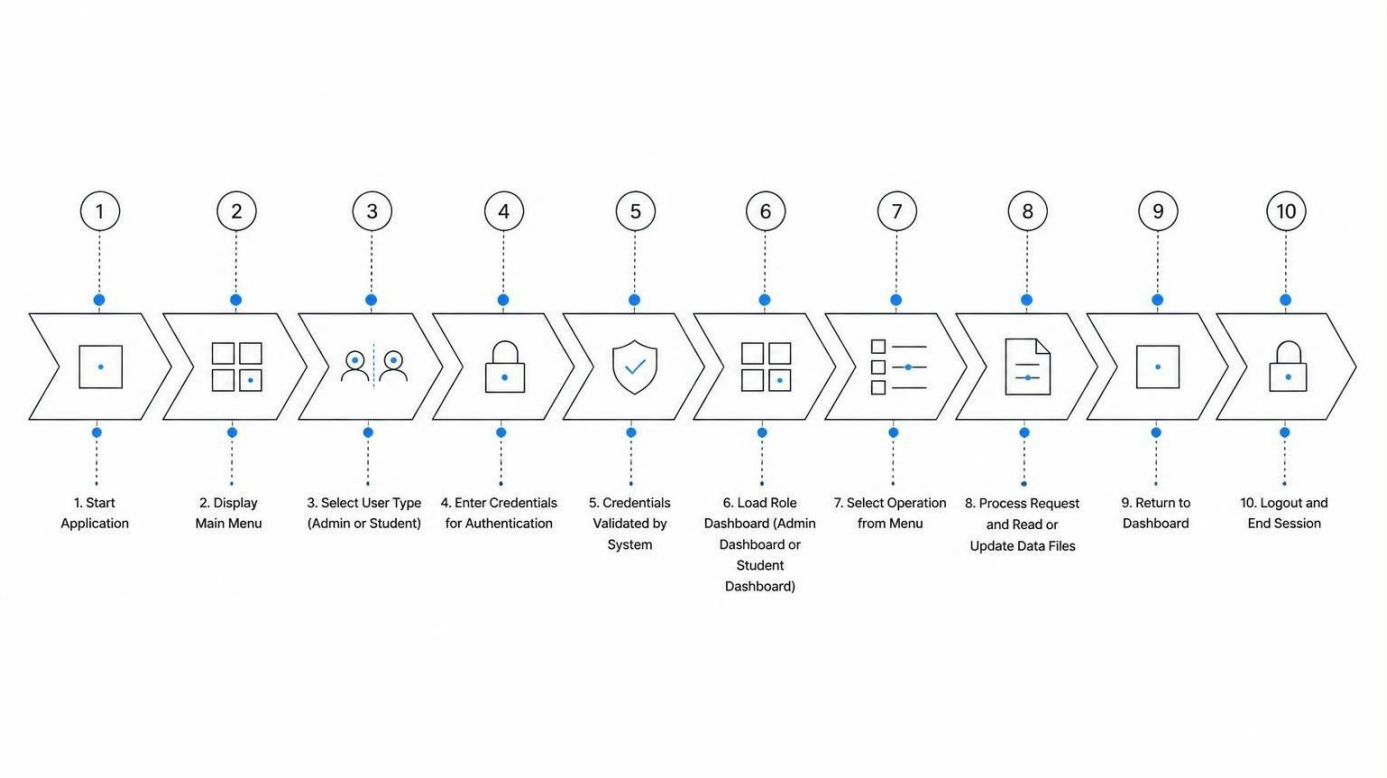
# Design Diagrams — Use Case & Workflow

## Use Case Diagram



The use case diagram illustrates the functional scope of SHMS across two authenticated roles. The Administrator has comprehensive management capabilities across all five domains, while the Student interface is restricted to self-service operations.

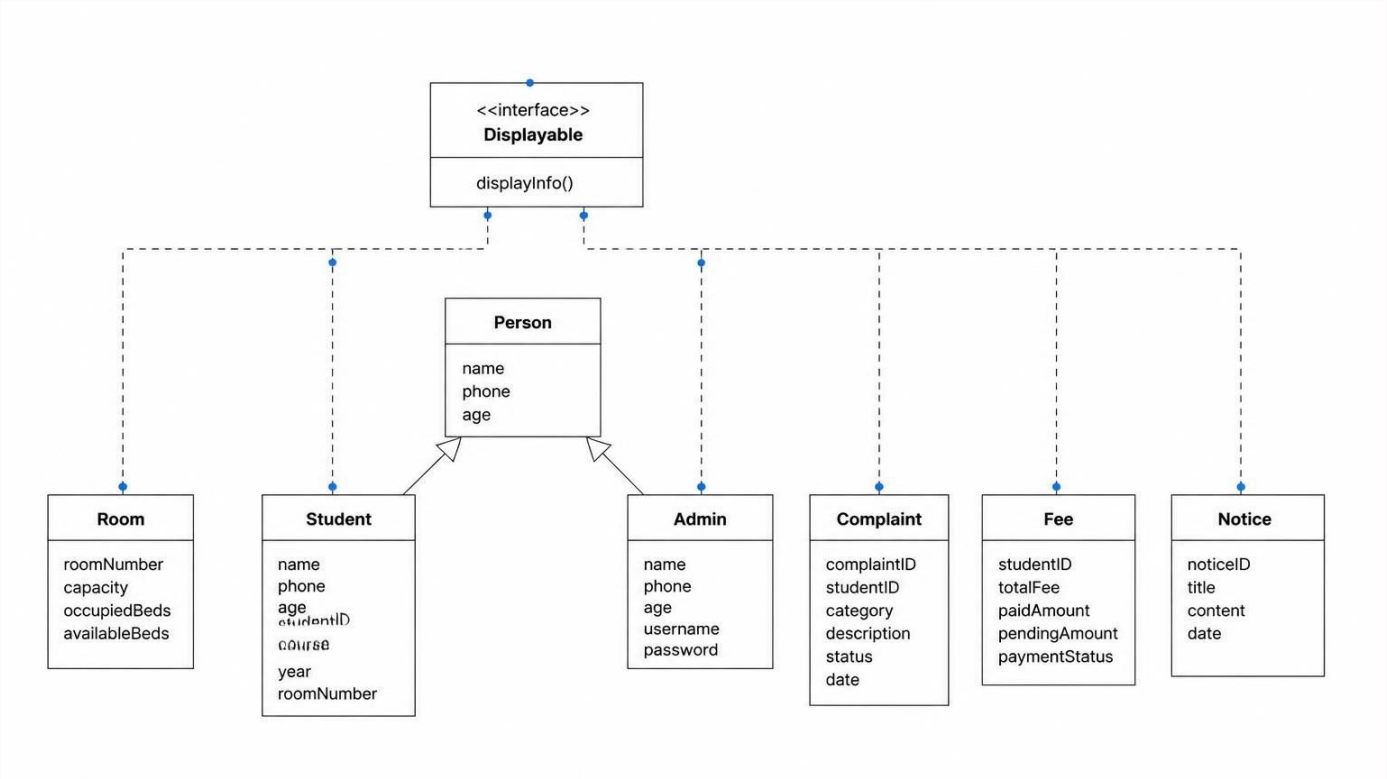
# Workflow Diagram



This workflow diagram illustrates the user journey within the Smart Hostel Management System, from initial application startup to the final logout. It details the sequential steps involved in user authentication, role-based dashboard access, execution of operations, and persistent data handling. The flow demonstrates how users interact with the system's console interface to manage and access hostel-related functionalities.

# Class & Component Diagram

This class diagram illustrates the key entities within the Smart Hostel Management System (SHMS), their attributes, and relationships. It highlights the use of inheritance for common properties and an interface for consistent display functionality across various components.



The diagram visually represents the object-oriented design principles applied, including inheritance from the **Person** class for both **Student** and **Admin** entities, and the implementation of the **Displayable** interface across several core classes to standardize information output.

# Implementation Details

## Java Concepts Demonstrated

### OOP

#### Foundations

Classes, objects, constructors, encapsulation with private attributes and public getters/setters

### Inheritance

Student extends Person , Admin extends Person — shared attributes defined once

### Interfaces

Displayable interface enforces displayInfo() across all entity classes

### Polymorphism

Method overriding for displayInfo() ; method overloading for student search by ID and by name

# Core Implementation & Data Structure

## Core Implementation Features

- ArrayList Collections: Dynamic in-memory storage loaded from text files at startup
- Exception Handling: Try-catch blocks manage IOException, NullPointerException, and input errors
- File Handling: BufferedReader / BufferedWriter for reading and writing delimited text records
- LocalDate: Java 8 date API for complaint timestamps and notice dates
- Control Statements: Switch-case menus, while-loop session control, if-else validation
- Smart Allocation: Iterates room list, checks availableBeds > 0, assigns first suitable room

## Data File Structure

| File           | Key Fields                                                        |
|----------------|-------------------------------------------------------------------|
| students.txt   | studentID   name   phone   age   course   year   roomNumber       |
| rooms.txt      | roomNumber   capacity   occupiedBeds   availableBeds              |
| complaints.txt | complaintID   studentID   category   description   status   date  |
| fees.txt       | studentID   totalFee   paidAmount   pendingAmount   paymentStatus |
| notices.txt    | noticeID   title   content   date                                 |

# Screenshots & Results

The following placeholders indicate key interface screens captured from the running Java console application. Replace each placeholder with actual screenshots taken during the final demonstration.

## Figure 1 — Main Menu

Initial console screen displaying user type selection: **1. Administrator Login, 2. Student Login, 3. Exit.**

```
=====
      SMART HOSTEL MANAGEMENT SYSTEM
=====
1. Admin Login
2. Student Login
3. Exit
Enter choice:
```

## Figure 2 — Admin Dashboard

Post-authentication menu showing all six management modules and the Reports option.

```
===== ADMIN LOGIN =====
Enter Admin Username: admin
Enter Admin Password: 1234

===== ADMIN DASHBOARD =====
1. Student Management
2. Room Management
3. Complaint Management
4. Fee Management
5. Notice Management
6. Reports
7. Logout
Enter choice: █
```

## Figure 3 — Student Management

Sub-menu for Add, Search, Update, Delete operations with sample student records displayed.

```
===== STUDENT MANAGEMENT =====
1. View All Students
2. Add Student
3. Search Student by ID
4. Search Student by Name
5. Update Student
6. Delete Student
7. Back
Enter choice: █
```

## Figure 4 — Smart Room Allocation

Output showing automatic room selection based on available bed count across all rooms.

```
===== ADD STUDENT =====
Student ID: 102
Name: Yash
Phone: 9099000882
Age: 20
Course: Btech CSE
Year: First
Invalid input! Please enter a whole number.
Year: 2026

Do you want to allocate a room now?
1. Yes - Smart Allocation
2. No
Enter choice: 1
Room A-101 allocated successfully!
Student added successfully!
```

# Screenshots & Results (Continued)

Figure 6 — Fee Management

Fee records showing total fee, paid amount, pending balance, and payment status per student.

```
===== FEE MANAGEMENT =====
1. View All Fee Records
2. Add Fee Record
3. Update Payment
4. Search Student Fee
5. Back
Enter choice: 1

===== ALL FEE RECORDS =====

Student ID   : 101
Total Fee    : ?60000.00
Paid         : ?40000.00
Pending      : ?20000.00
Status       : PENDING
```

Figure 7 — Student Dashboard

Student-side menu after login: View Profile, View Room, Submit Complaint, View Fees, Make Payment, View Notices.

```
===== STUDENT DASHBOARD =====
Welcome, Rahul!
1. View Profile
2. View Room Details
3. Submit Complaint
4. View My Complaints
5. View My Fee Status
6. Make Fee Payment
7. View Notices
8. Logout
Enter choice: █
```

# Testing Approach

| Test Case               | Input/Scenario                     | Expected Result                            | Status            |
|-------------------------|------------------------------------|--------------------------------------------|-------------------|
| Valid Admin Login       | Username: admin,<br>Password: 1234 | Access granted, Admin<br>Dashboard loads   | Pass              |
| Invalid Admin<br>Login  | Incorrect password                 | Error message, login<br>retry prompted     | pass              |
| Student Login           | Valid Student ID                   | Access granted, Student<br>Dashboard loads | pass              |
| Add Student             | Unique ID, valid details           | Record added,<br>confirmation displayed    | Pass              |
| Duplicate Student<br>ID | Existing Student ID                | Duplicate error, record<br>rejected        | To be<br>verified |
| Search Student          | Valid ID or name<br>substring      | Matching record(s)<br>displayed            | pass              |

# Testing Approach

| Test Case             | Input/Scenario                    | Expected Result                        | Status |
|-----------------------|-----------------------------------|----------------------------------------|--------|
| Update Student        | Valid ID, modified fields         | Record updated, confirmation shown     | Pass   |
| Delete Student        | Valid existing Student ID         | Record removed, confirmation shown     | Pass   |
| Smart Room Allocation | Student with no room assigned     | First room with available bed assigned | Pass   |
| Full Room Allocation  | All rooms at capacity             | Appropriate message, no allocation     | Pass   |
| Vacate Room           | Valid Student ID with room        | Room vacated, bed count updated        | Pass   |
| Invalid Numeric Input | Non-numeric where number expected | Handled gracefully, re-prompt shown    | Pass   |

# Testing Approach

| Test Case               | Input/Scenario                          | Expected Result                     | Status         |
|-------------------------|-----------------------------------------|-------------------------------------|----------------|
| Submit Complaint        | Category, description                   | Complaint saved with Pending status | Pass           |
| Update Complaint Status | Select Pending → In Progress / Resolved | Status updated, change reflected    | Pass           |
| Valid Fee Payment       | Amount $\leq$ pending balance           | Payment recorded, balance updated   | Pass           |
| Invalid Payment         | Amount $>$ pending balance              | Error message, transaction rejected | Pass           |
| Add Notice              | Title and content                       | Notice saved, visible to students   | To be verified |
| Generate Report         | Select Reports from Admin menu          | Consolidated statistics displayed   | To be verified |

# Challenges Faced

01

---

**Single-File Class Organization** - Managing multiple classes in one Main.java required careful structuring using static nested classes and modular methods.

02

---

**Text File Synchronization** - Ensuring file writes complete before subsequent reads required explicit flush and close operations on writers.

03

---

**Smart Allocation Logic** - Iterating the room list and correctly updating bed counts without race conditions required careful loop design.

04

---

**Input Validation** - Handling non-numeric inputs, empty strings, and edge cases required comprehensive try-catch and validation checks.

## Learnings, Future Enhancements & References

### Learnings & Key Takeaways

- **OOP in Practice:** Inheritance, interfaces, and encapsulation translate directly to maintainable code structure.
- **File-Based Persistence:** Text files provide a lightweight alternative to databases for small-to-medium datasets.
- **Defensive Programming:** Validating all user inputs and handling exceptions prevents unexpected crashes.
- **Modular Design:** Decomposing logic into discrete methods improves readability and testability.
- **Role-Based Access:** Separating Admin and Student operations enforces security boundaries at the application level.
- **Smart Algorithms:** Even simple allocation logic demonstrates how automation reduces manual effort.

# Future Enhancements & References

## Future Enhancements

- Migrate from text files to MySQL with JDBC for robust data management
  - Develop a JavaFX or Swing GUI for improved user experience
  - Add email notifications for complaint status updates and fee reminders
  - Implement roommate matching based on course and year preferences
  - Add data export to PDF and Excel for administrative reporting
  - Introduce multi-level authentication with password hashing
  - Deploy as a web application using Spring Boot for remote access
- 

## References

1. [1] Oracle. (2024). Java SE Documentation. Oracle Corporation. <https://docs.oracle.com/en/java/>
  2. [2] Horstmann, C. S. (2022). Core Java Volume I — Fundamentals (12th ed.). Pearson Education.
  3. [3] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
  4. [4] Pressman, R. S. (2014). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill.
  5. [5] JavaTPoint. (2024). Java File Handling. <https://www.javatpoint.com/java-file-handling>
- 

The Smart Hostel Management System demonstrates practical application of Java OOP principles in a console-based, file-persisted environment. It addresses real administrative challenges through structured design, role-based access, and automated operations — serving as a strong foundation for future enhancements toward a full production system.